3. ESTRATEGIAS DE BÚSQUEDA INFORMADAS

(PARTE 2)

IA 3.2 - Programación III

1° C - 2026

Dr. Mauro Lucci

➡️ Repaso

¿Cómo construir heurísticas? 🤔

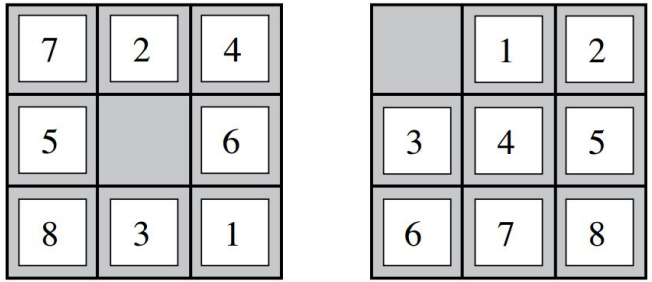
Motivación

Juego del 8 (8 Puzzle)

7	2	4
5		6
8	3	1

	1	2
3	4	5
6	7	8

Descripción

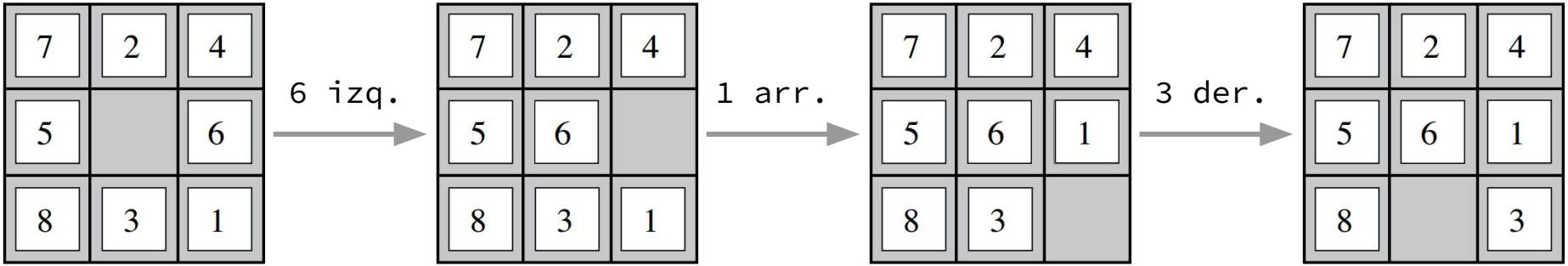


Objetivo. Alcanzar una configuración objetivo en el menor número de deslizamientos.

Reglas. Una ficha se puede mover de una casilla *A* a una casilla *B* si:

1. *A* está adyacente (verticalmente u horizontalmente) a *B*, y
2. *B* está vacía.

📖 Ejemplo



📝 Propuesta de formulación

Un **estado** lo representamos con una matriz 3x3 donde cada casilla tiene un número distinto de 0 a 8 (0 representa a la casilla vacía).

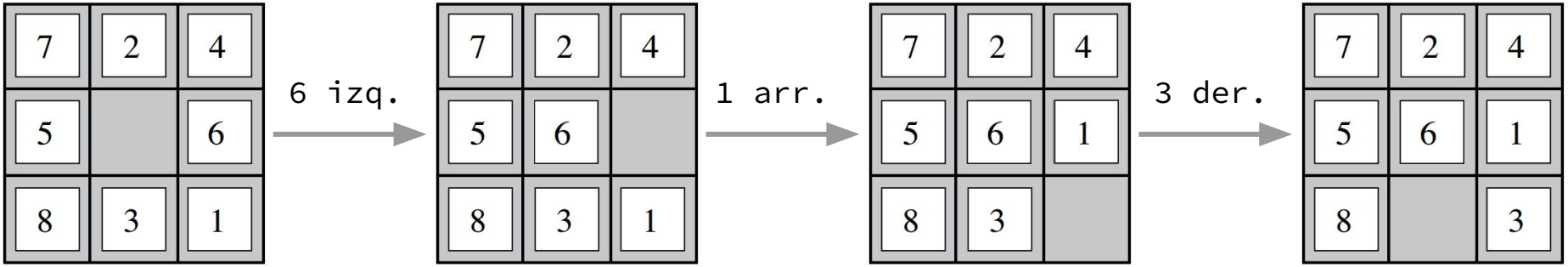
El número total de estados es 9! = 362880 estados, de los cuales solo la mitad (181440) son alcanzables.

Una **acción** la representamos con una dirección:

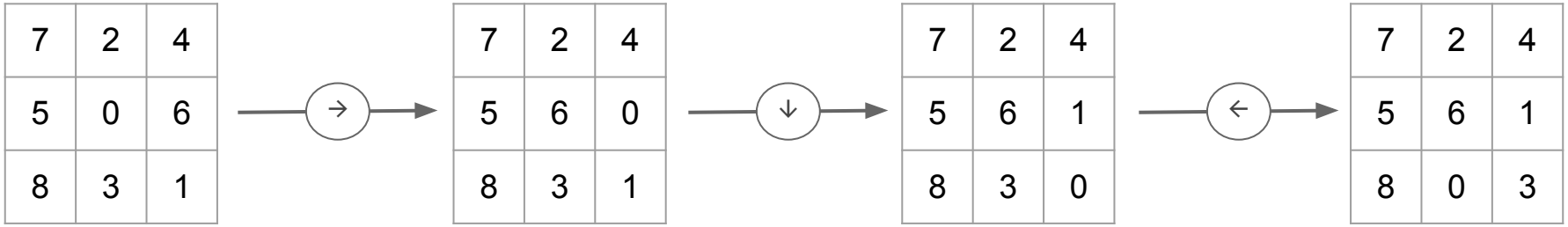
←, ↑, →, ↓

que indica hacia dónde se mueve el 0.

📖 Ejemplo



Representación:



📝 Propuesta de formulación

- **Estado inicial.** Cualquier estado puede ser designado como inicial.
- **Acciones.** Dado un estado M,

$$
\text{ACCIONES}(M) = \begin{cases}
\{\rightarrow,\downarrow\} & \text{si } M[1][1] = 0 \\
\{\rightarrow,\downarrow,\leftarrow\} & \text{si } M[1][2] = 0 \\
\{\downarrow,\leftarrow\} & \text{si } M[1][3] = 0 \\
\{\rightarrow,\downarrow,\uparrow\} & \text{si } M[2][1] = 0 \\
\{\rightarrow,\downarrow,\leftarrow,\uparrow\} & \text{si } M[2][2] = 0 \\
\{\downarrow,\leftarrow,\uparrow\} & \text{si } M[2][3] = 0 \\
\{\rightarrow,\uparrow\} & \text{si } M[3][1] = 0 \\
\{\rightarrow,\leftarrow,\uparrow\} & \text{si } M[3][2] = 0 \\
\{\leftarrow,\uparrow\} & \text{si } M[3][3] = 0
\end{cases}
$$

Formulación

Notación: Dado un estado M, notamos con i_0 y j_0 a la fila y columna de M donde se ubica el 0, es decir, tales que $M[i_0][j_0] = 0$.

- **Modelo de transiciones.** Dado un estado M,

  RESULTADO(M,→) = M', donde para todo $i,j \in \{1,2,3\}$

$$M'[i][j] = \begin{cases} M[i][j+1] & \text{si } (i = i_0) \wedge (j = j_0) \\ 0 & \text{si } (i = i_0) \wedge (j = j_0+1) \\ M[i][j] & \text{en caso contrario} \end{cases}$$

  Queda completar como ejercicio el resultado de aplicar las otras acciones.

- **Test objetivo.** Verificar si el estado coincide con la configuración deseada.
- **Costo de camino.** El costo de camino es la suma de los costos individuales.

  Dado un estado M y una acción a, COSTO-INDIVIDUAL(M,a) = 1.

Solver

Solver para el **8-puzzle** que implementa una **búsqueda de grafo** con diferentes **estrategias de búsqueda**.

https://deniz.co/8-puzzle-solver/

Probar por ejemplo con la entrada 123608547.

Resolución mediante búsqueda

Dado un estado inicial generado aleatoriamente.

- Menor profundidad de un nodo objetivo:

  ***d* = 22** (en promedio)

- Factor de ramificación:

  ***b* = 3** (en promedio)

- El árbol de búsqueda hasta el nivel 22 tiene aproximadamente:

  $3^{22+1} - 1 \approx 94.143.178.826 \approx$ **$9{,}41 \times 10^{10}$ nodos**

- En general, no es posible resolver este problema con TREE-BFS.

Resolución mediante búsqueda

- El número de estados alcanzables es **181440**, que es muchísimo menor a **9,41 x 10^{10}** (número promedio de nodos generados por TREE-BFS).
- La diferencia radica en la enorme cantidad de caminos redundantes.
- Luego, es necesario pasar de TREE-BFS a GRAPH-BFS.

Resolución mediante búsqueda

- En puzzle más grandes, el número de estados alcanzables puede ser demasiado grande.
- Por ejemplo, el juego del 15 (15 puzzle) tiene $\mathbf{10^{13}}$ estados.
- Luego, es necesario pasar de GRAPH-BFS a GRAPH-ASTAR, lo que involucra disponer de una **buena heurística**.

Heurísticas

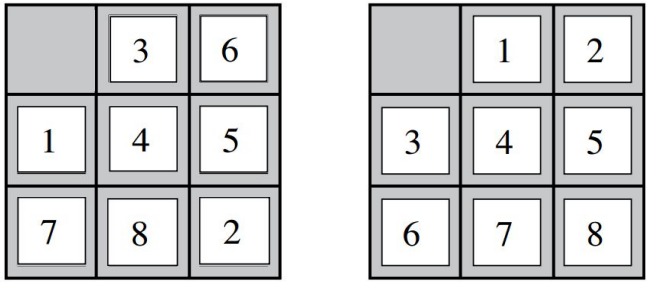
Veamos dos ejemplos de heurísticas para el 8-puzzle.

Heurísticas

---

h_1: número de fichas (numéricas) mal ubicadas.

📖 **Ejemplo.** Sea un nodo *n* con el estado de la izquierda y sea el estado objetivo de la derecha.

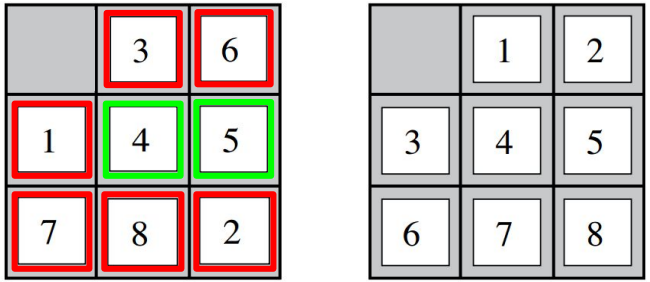


Heurísticas

---

h_1: número de fichas (numéricas) mal ubicadas.

📖 **Ejemplo.** Sea un nodo *n* con el estado de la izquierda y sea el estado objetivo de la derecha.



$h_1(n) = 6$

Distancia de Manhattan

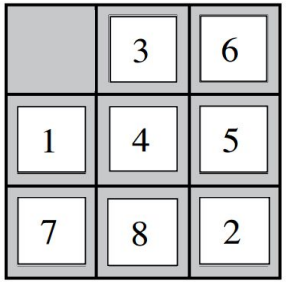
En rojo se representa la **distancia en línea recta** (o euclídea) y en azul y verde la **distancia de Manhattan.**

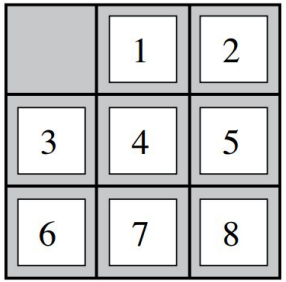
Heurísticas

---

h_2: suma de la **distancia de Manhattan** de cada ficha (numérica) a su posición correcta.

📖 **Ejemplo.** Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.



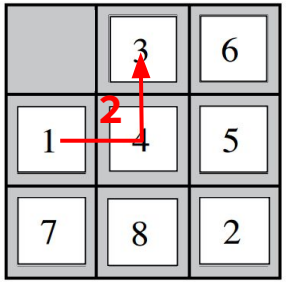


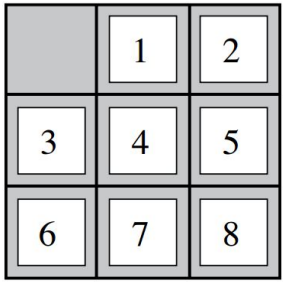
Heurísticas

---

h_2: suma de la **distancia de Manhattan** de cada ficha (numérica) a su posición correcta.

📖 **Ejemplo.** Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.





h_2(n) = **2**

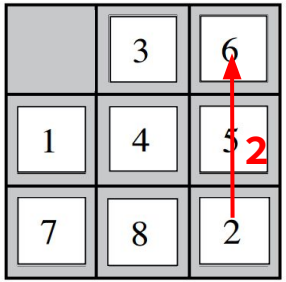
	↑	↑	↑	↑	↑	↑	↑	↑
Ficha:	1	2	3	4	5	6	7	8

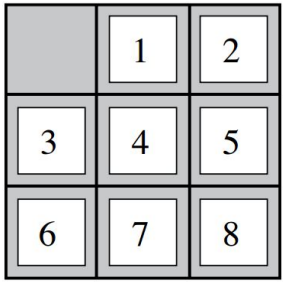
Heurísticas

---

h_2: suma de la **distancia de Manhattan** de cada ficha (numérica) a su posición correcta.

📖 **Ejemplo.** Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.





$h_2(n) = 2 + \mathbf{2}$

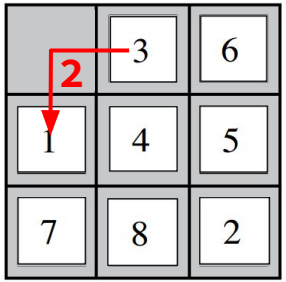
	↑	↑	↑	↑	↑	↑	↑	↑
Ficha:	1	2	3	4	5	6	7	8

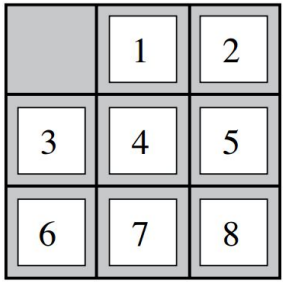
Heurísticas

---

h_2: suma de la **distancia de Manhattan** de cada ficha (numérica) a su posición correcta.

📖 **Ejemplo.** Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.





$h_2(n) = 2 + 2 + \mathbf{2}$

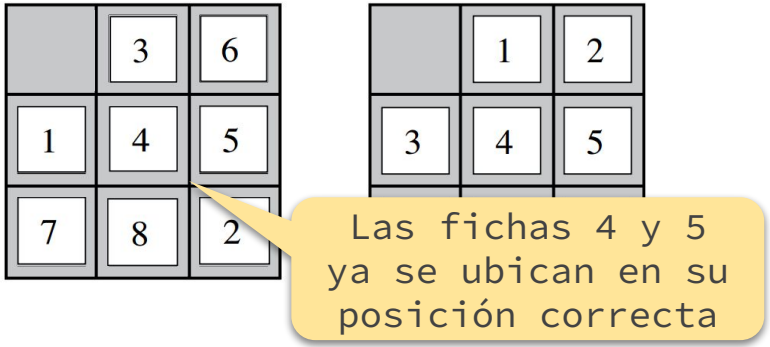
	↑	↑	↑	↑	↑	↑	↑	↑
Ficha:	1	2	3	4	5	6	7	8

Heurísticas

---

h_2: suma de la **distancia de Manhattan** de cada ficha (numérica) a su posición correcta.

📖 **Ejemplo.** Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.



$h_2(n) = 2 + 2 + 2 + 0 + 0$

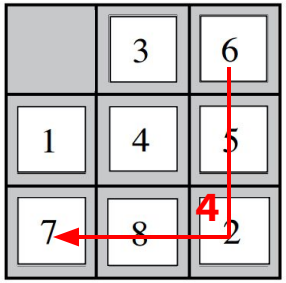
	↑	↑	↑	↑	↑	↑	↑	↑
Ficha:	1	2	3	4	5	6	7	8

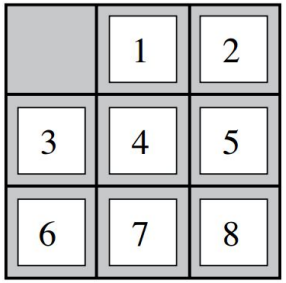
Heurísticas

---

h_2: suma de la **distancia de Manhattan** de cada ficha (numérica) a su posición correcta.

📖 **Ejemplo.** Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.





$h_2(n) = 2 + 2 + 2 + 0 + 0 + \mathbf{4}$

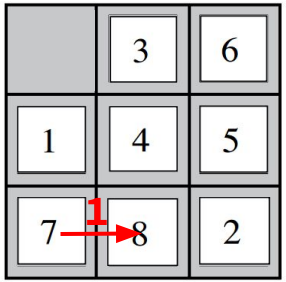
	↑	↑	↑	↑	↑	↑	↑	↑
Ficha:	1	2	3	4	5	6	7	8

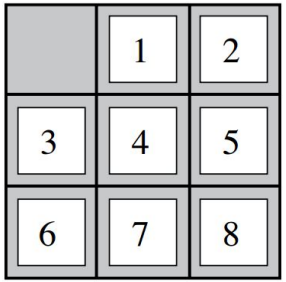
Heurísticas

---

h_2: suma de la **distancia de Manhattan** de cada ficha (numérica) a su posición correcta.

📖 **Ejemplo.** Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.





$h_2(n) = 2 + 2 + 2 + 0 + 0 + 4 + \mathbf{1}$

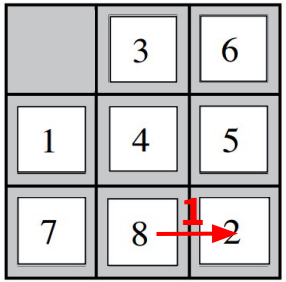
	↑	↑	↑	↑	↑	↑	↑	↑
Ficha:	1	2	3	4	5	6	7	8

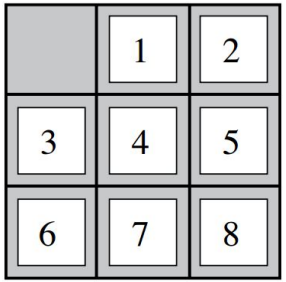
Heurísticas

---

h_2: suma de la **distancia de Manhattan** de cada ficha (numérica) a su posición correcta.

📖 **Ejemplo.** Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.





$h_2(n) = 2 + 2 + 2 + 0 + 0 + 4 + 1 + \mathbf{1} = \mathbf{12}$

	↑	↑	↑	↑	↑	↑	↑	↑
Ficha:	1	2	3	4	5	6	7	8

📝 Ejercicio

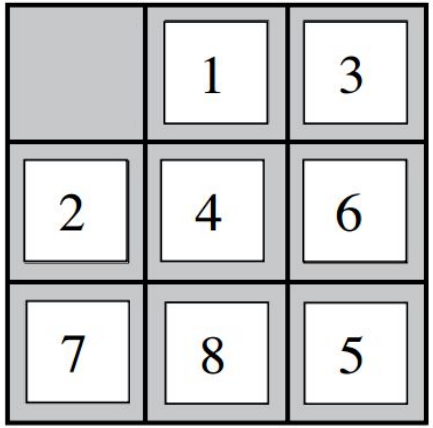
Sea un nodo n con el estado de la izquierda y sea el estado objetivo de la derecha.

	1	3
2	4	6
7	8	5

	1	2
3	4	5
6	7	8

Calcular $h_1(n)$ (heurística de fichas mal ubicadas) y $h_2(n)$ (heurística de distancias de Manhattan).

Respuesta



h_1(n) = 6 (solamente las fichas 1 y 4 están bien ubicadas).

h_2(n) = 0 + 3 + 3 + 0 + 1 + 3 + 1 + 1 = 12

	↑	↑	↑	↑	↑	↑	↑	↑
Ficha:	1	2	3	4	5	6	7	8

h_1 vs. h_2

d	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6
4	112	13	12
6	680	20	18
8	6384	39	25
10	47127	93	39
12	3644035	227	73
14	–	539	113
16	–	1301	211
18	–	3056	363
20	–	7276	676
22	–	18094	1219
24	–	39135	1641

Observación 1.
Se generan muchos menos nodos con las estrategias de búsqueda informadas.

Observación 2.
La heurística h_2 genera menos nodos que la h_1.
¿Por qué? 🤔

Cada fila de la tabla reporta el número promedio de nodos generados por cada estrategia (en búsqueda de árbol) para 100 instancias aleatorias de 8 puzzle con solución óptima en la profundidad d.

h_1 vs. h_2

- Se puede probar que ambas son **admisibles**.
- ¿h_2 es siempre mejor que h_1? **Si**.
- **Definición.** Sean dos heurísticas h_1 y h_2. Se dice que h_2 **domina** a h_1 si, para todo nodo n,

$$h_2(n) \geq h_1(n)$$

- **Teorema.** Sean dos heurísticas h_1 y h_2 tales que h_2 domina a h_1, **A* con h_2 nunca expande más nodos que A* con h_1**.

¿Cómo generar heurísticas de este tipo? 🤔🤔

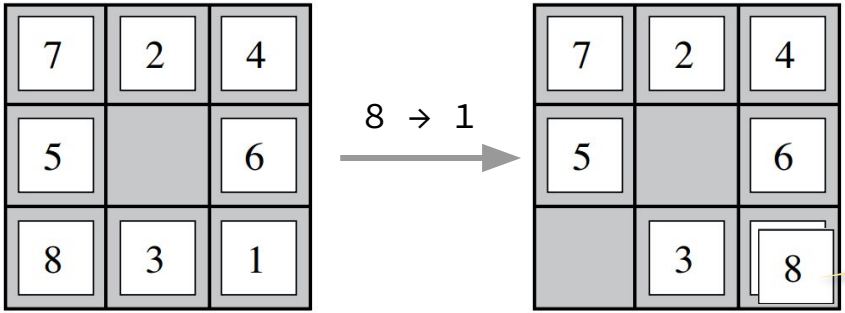
Problemas relajados

Un **problema relajado** se obtiene quitando restricciones a las acciones de un problema.

📖 Ejemplo - Relajación Nº 1

Una ficha se puede mover de una casilla *A* a una casilla *B* ~~si:~~

~~1. *A* está adyacente (verticalmente u horizontalmente) a *B*, y~~
~~2. *B* está vacía.~~

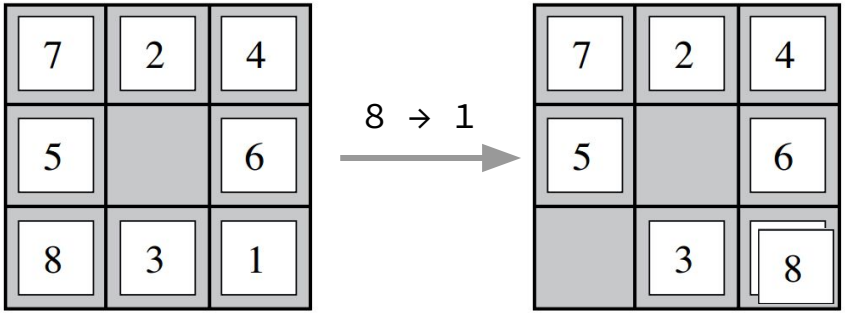


Ahora dos fichas pueden compartir la misma casilla.

📖 Ejemplo - Relajación Nº 1

Una ficha se puede mover de una casilla *A* a una casilla *B* ~~si:~~

~~1. *A* está adyacente (verticalmente u horizontalmente) a *B*, y~~
~~2. *B* está vacía.~~

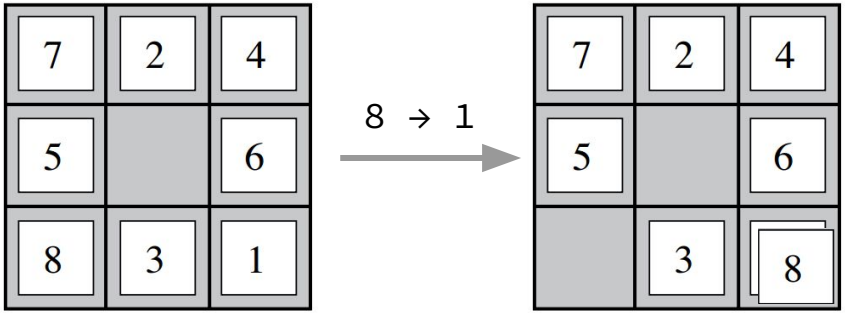


¿Cuántos movimientos se necesitan para resolverlo? 🤔

📖 Ejemplo - Relajación Nº 1

Una ficha se puede mover de una casilla *A* a una casilla *B* ~~si:~~

~~1. *A* está adyacente (verticalmente u horizontalmente) a *B*, y~~
~~2. *B* está vacía.~~



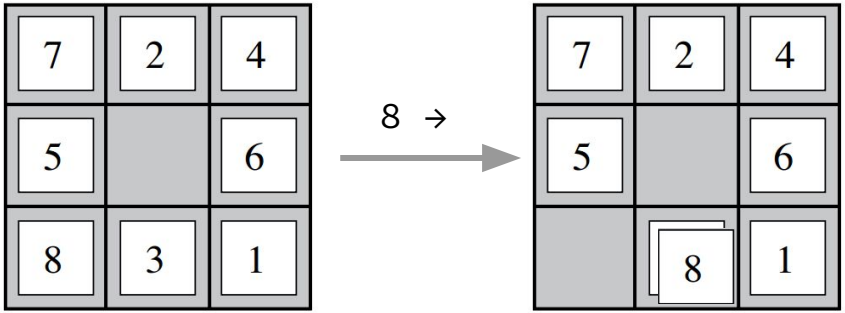
¿Cuántos movimientos se necesitan para resolverlo? 🤔

Uno por cada ficha numérica mal ubicada.

📖 Ejemplo - Relajación Nº 2

Una ficha se puede mover de una casilla *A* a una casilla *B* si:

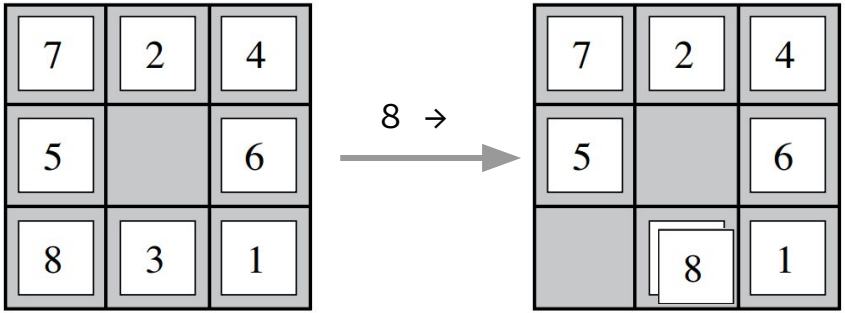
1. *A* está adyacente (verticalmente u horizontalmente) a *B*, ~~y~~
~~2. *B* está vacía.~~



📖 Ejemplo - Relajación Nº 2

Una ficha se puede mover de una casilla *A* a una casilla *B* si:

1. *A* está adyacente (verticalmente u horizontalmente) a *B*, ~~y~~
~~2. *B* está vacía.~~

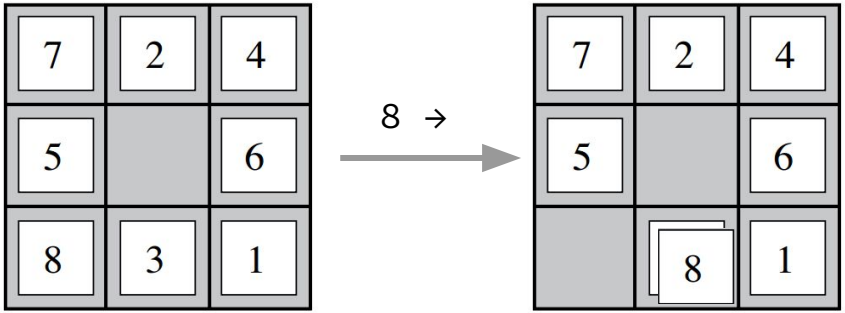


¿Cuántos movimientos se necesitan para resolverlo? 🤔

📖 Ejemplo - Relajación Nº 2

Una ficha se puede mover de una casilla *A* a una casilla *B* si:

1. *A* está adyacente (verticalmente u horizontalmente) a *B*, ~~y~~
~~2. *B* está vacía.~~



¿Cuántos movimientos se necesitan para resolverlo? 🤔

Tantos como la distancia de Manhattan entre cada ficha numérica mal ubicada y su posición correcta.

Propiedades

Sea *P'* un problema relajado de un problema *P*.

1. El grafo de espacio de estados de *P* es un **subgrafo** del de *P'*.
2. El costo de una solución óptima en *P'* es una **cota inferior** del costo de una solución óptima en P.
3. Si *P'* **se resuelve sin búsqueda** (por ej. con una fórmula cerrada), podemos derivar una **heurística** para *P*.

Relajación Nº1 → Heurística h_1.

Relajación Nº2 → Heurística h_2.

Admisibilidad y consistencia

Proposición. Sean *P’* un problema relajado de un problema *P* y *h* la heurística derivada de *P’*, entonces *h* es **consistente** (y por ende, también **admisible**) para *P*.

Subproblema

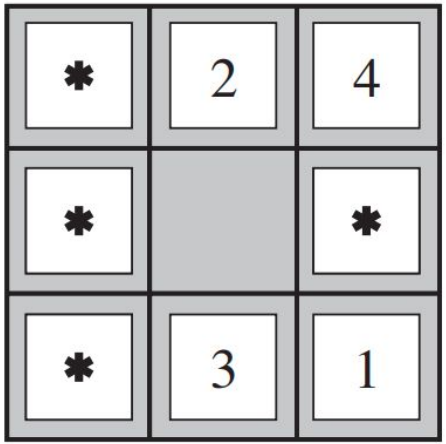
Un **subproblema** consiste en resolver una parte de un problema más grande.

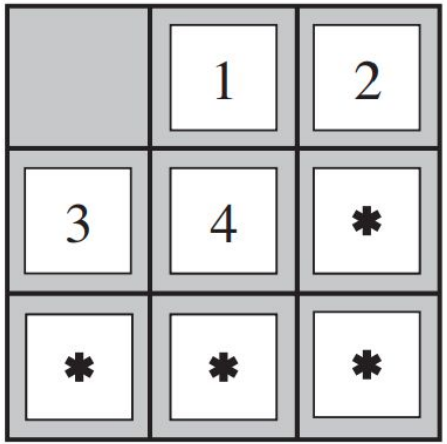
En general, se obtiene quitando restricciones al objetivo de un problema.

📖 Ejemplo

En lugar de ubicar todas las fichas en su posición correcta, pensamos en el subproblema cuyo objetivo es ubicar únicamente a las fichas 1-2-3-4 y la casilla vacía en su posición correcta, respetando las reglas de movimiento del problema original.

Las demás fichas las podemos reemplazar con una *, porque no interesan quiénes son.





La elección del subproblema 1-2-3-4 es arbitraria. Podría ser 5-6-7-8, 2-4-6-8, etc.

📖 Ejemplo

En el subproblema 1-2-3-4, los siguientes estados son objetivos:

	1	2
3	4	5
6	7	8

	1	2
3	4	6
5	7	8

	1	2
3	4	8
7	6	5

, etc.

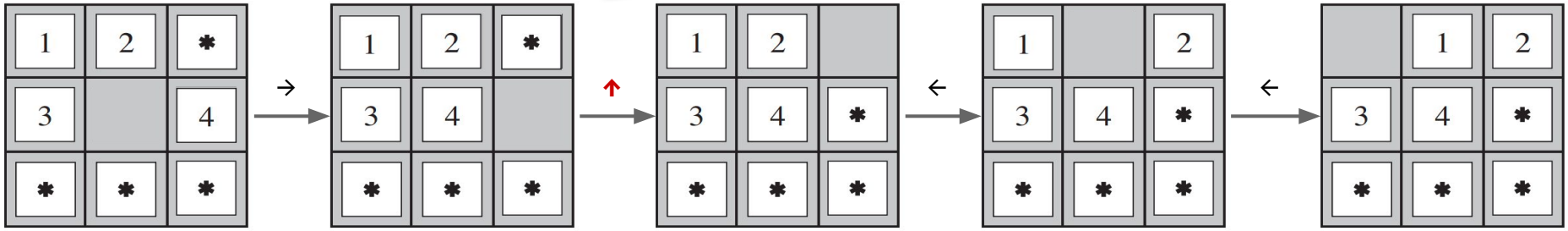
Y todos ellos se pueden representar con el siguiente:

	1	2
3	4	*
*	*	*

Notar que los últimos dos estados no son objetivos en el problema original.

📖 Ejemplo

Aquí movemos una ficha *, pero sigue contando como una acción.



Estado inicial

Estado objetivo

Esta solución tiene costo de camino 4 en el subproblema 1-2-3-4 y es óptima.

Propiedades

Sea un subproblema *P'* de un problema *P*,

1. El costo de una solución óptima de *P'* es una **cota inferior** del costo de una solución óptima de *P*.

   📖 **Ejemplo.** Luego de resolver el subproblema 1-2-3-4, aún quedan por ubicar las demás fichas 5,6,7,8 (si es que no están ya en su posición correcta).

2. Si *P'* **se resuelve rápidamente con una búsqueda** (por ej. si su espacio de estados es mucho más chico), podemos derivar una **heurística** para *P*.

Base de datos de patrones – Pattern database

Una **base de dato de patrones** precalcula el costo de una solución óptima **para cualquier estado inicial de un subproblema** y lo almacena en memoria (en un diccionario).

Luego, al resolver el problema original, para computar $h(n)$ basta con buscar en la base de datos de patrones el costo correspondiente.

Espacio de estados

En el 8-puzzle el número de estados alcanzables es:

9!/2 = **181440**

En el subproblema 1-2-3-4, representando a las fichas 5-6-7-8 con *, el número de estados alcanzables es:

15120

El espacio de estados es **12 veces más chico**.

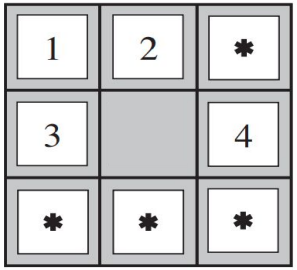
Una base de datos de patrones para el subproblema 1-2-3-4 tendrá 15120 entradas.

📖 Ejemplo

---

1. Para **cada estado inicial**, resolvemos el subproblema 1-2-3-4 **con búsqueda** y guardamos el costo de una solución óptima en la base de datos de patrones.

s_1 =



: 4

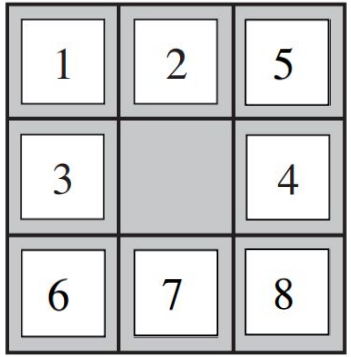
… y así sucesivamente con todos los estados iniciales.

📖 Ejemplo

---

2. Sea *n* un nodo con el estado *s*, ¿cómo calculamos h(s)?

s =



📖 Ejemplo

---

2. Sea *n* un nodo con el estado *s*, ¿cómo calculamos h(s)?

s =

1	2	5
3		4
6	7	8

s’ =

1	2	*
3		4
*	*	*

: 4

3. Determinamos su estado asociado s’ en el subproblema 1-2-3-4 y lo buscamos en la base de datos. Luego, h(n) = 4.

Admisibilidad

Proposición. Sean *P’* un subproblema de un problema *P* y *h* la heurística derivada de *P’*, entonces *h* es **admisible** para *P*.

Ejercicio

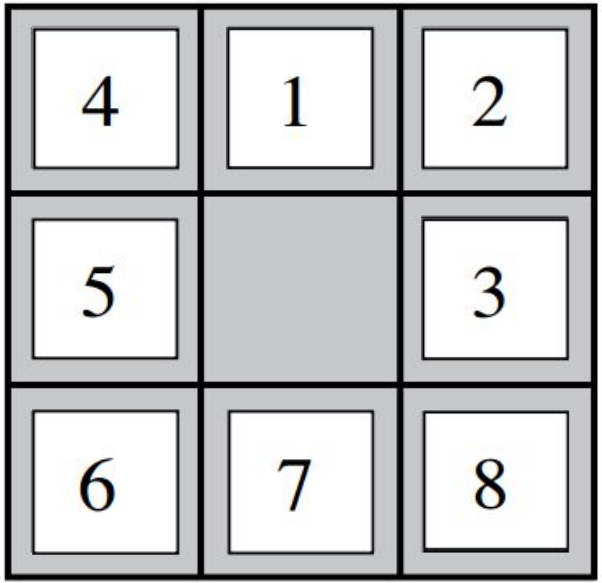
Considerar el estado de la izquierda y sea el estado objetivo de la derecha.

4	1	2
5		3
6	7	8

	1	2
3	4	5
6	7	8

¿Qué heurística da la mejor estimación para este estado particular: fichas mal ubicadas, distancias de manhattan o 2-3-4-5?

Respuesta



Cantidad de fichas numéricas mal ubicadas: **3**

Distancias de manhattan:

0 + 0 + 2 + 2 + 2 + 0 + 0 + 0 = **6**

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

1 2 3 4 5 6 7 8

Respuesta

4	1	2
5		3
6	7	8

4	*	2
5	0	3
*	*	*

→

4	*	2
5	3	0
*	*	*

→

4	*	0
5	3	2
*	*	*

→

4	0	*
5	3	2
*	*	*

→

4	3	*
5	0	2
*	*	*

→

4	3	*
0	5	2
*	*	*

↘

0	3	*
4	5	2
*	*	*

←

3	0	*
4	5	2
*	*	*

←

3	*	0
4	5	2
*	*	*

←

3	*	2
4	5	0
*	*	*

←

3	*	2
4	0	5
*	*	*

←

3	*	2
0	4	5
*	*	*

←

0	*	2
3	4	5
*	*	*

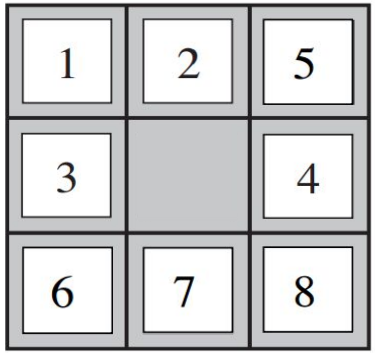
En el subproblema 2-3-4-5 se necesitan **12** acciones.

Fortalezas y debilidades

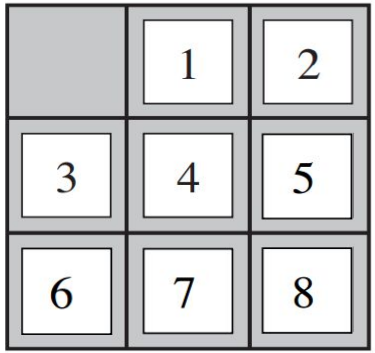
- Dependiendo del estado, la heurística basada en la base de datos de patrones puede ofrecer una **estimación más precisa** que la heurística de fichas mal ubicadas y la heurística de distancias de Manhattan.
- La heurística 1-2-3-4 sigue siendo una **estimación débil**, ya que no exige que las demás fichas numéricas estén correctamente posicionadas.
- ¿Podemos **sumarla** con la heurística de la base de datos 5-6-7-8? 🤔

📖 Ejemplo

Considerar el siguiente 8-puzzle:



Estado inicial



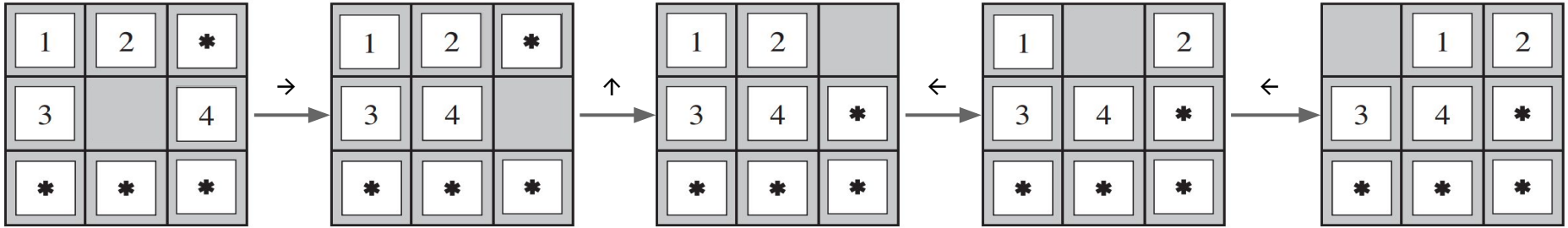
Estado objetivo

¿Cuáles son los respectivos costos de camino de una solución óptima en los subproblemas 1-2-3-4 y 5-6-7-8?

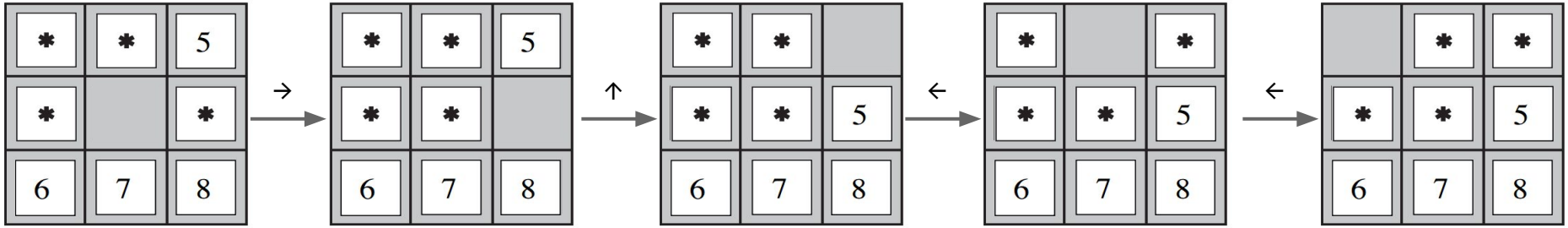
📖 Ejemplo

¿La suma de ambos nos dará el costo de una solución óptima en el problema original? 🤔

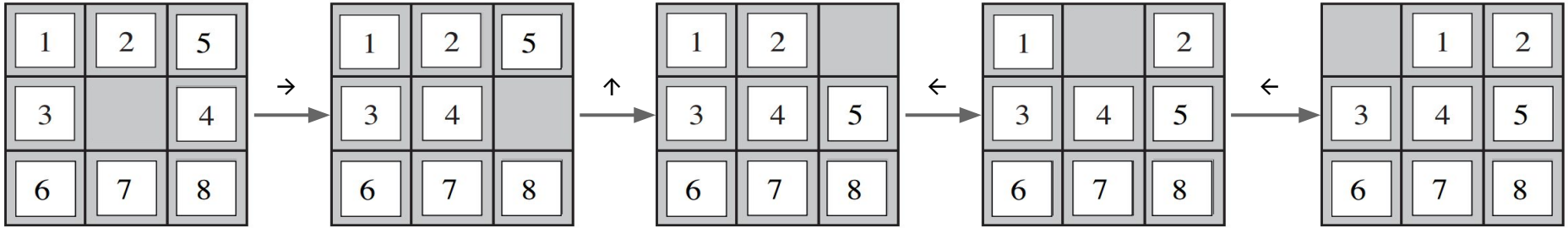
En el subproblema 1-2-3-4, el costo de camino de una solución óptima es 4.



En el subproblema 5-6-7-8, el costo de camino de una solución óptima es 4.



📖 Ejemplo



En el problema original, el costo de camino de una solución óptima es

4 < 8 = 4 + 4

Al sumarlos directamente podemos sobrestimar

Combinar heurísticas

- ¿Las heurísticas obtenidas de las bases de datos 1-2-3-4 y 5-6-7-8 se pueden sumar?

  No directamente…

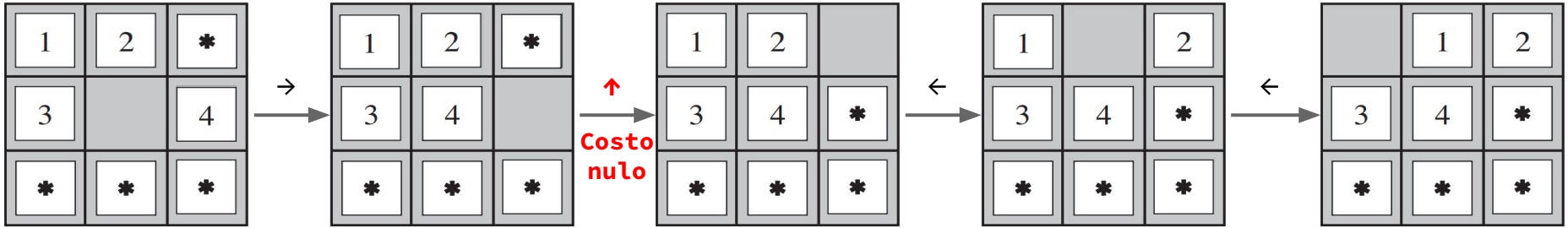
  Las soluciones de los subproblemas pueden **compartir** acciones, las cuales se contarían más de una vez.

- Para sumarlas, no se deben contar los movimientos que desplacen a una ficha *, es decir, esas acciones deben considerarse con costo individual **nulo**.

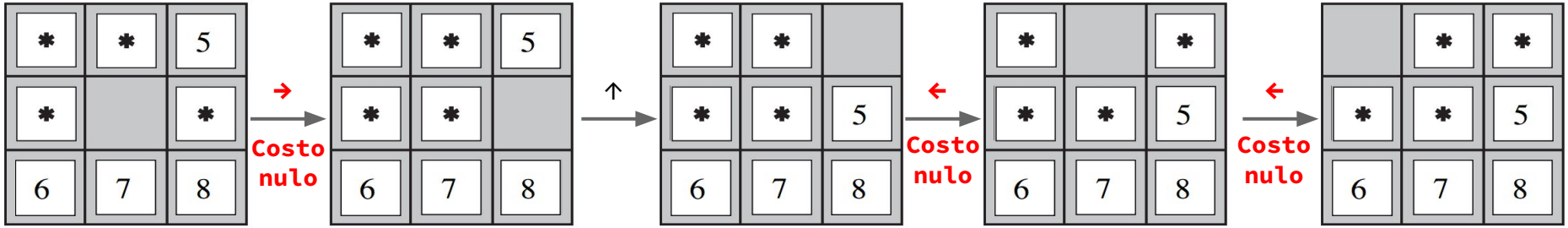
📖 Ejemplo

Ahora sí, al sumarlas obtenemos 3+1 = 4, que es el costo de camino de una solución óptima en el problema original. No siempre obtendremos el costo exacto (¡es una heurística!), pero de seguro no sobrestima.

En el subproblema 1-2-3-4, el costo de camino de una solución óptima es **3**.



En el subproblema 5-6-7-8, el costo de camino de una solución óptima es **1**.



Base de datos de patrones disjuntos

- “Sumando” (sin contar movimientos de fichas con *) las heurísticas de dos bases de datos de patrones **disjuntos** se obtiene una heurística **admisible** para el problema original.
- Esta técnica se conoce como **base de datos de patrones disjuntos**.
- Permiten resolver el 15-puzzle en milisegundos.

Heurística compuesta

Sean $h_1,\ldots,h_m$ heurísticas para un problema y tales que ninguna domina la otra.

Se define la **heurística compuesta**

$$h(n) = \max\{h_1(n),\ldots,h_m(n)\}.$$

Admisibilidad y consistencia

Proposiciones.

- Si $h_1,\ldots,h_m$ son admisibles, entonces h es **admisible**.
- Si $h_1,\ldots,h_m$ son consistentes, entonces h es **consistente**.
- h **domina** a $h_1,\ldots,h_m$.

📚 Resumen

- Vimos 3 formas de construir buenas heurísticas.
  a. Relajando el problema de forma tal que se pueda resolver sin búsqueda (con una fórmula cerrada).
  b. Resolviendo un subproblema para todos sus estados iniciales y almacenando en una base de datos de patrones el valor de una solución óptima.
  c. Componiendo heurísticas disponibles.

➡️ Próximamente

En algunos problemas, nuestras decisiones están afectadas por las acciones de **otros agentes** que **compiten** en nuestra contra.

Veremos problemas y algoritmos de **búsqueda entre adversarios**.